Multi-Campus University Management System

By
Ishita Nakhawa
Roll No: 150096725213

Problem Statement

The Challenge: Managing multiple university campuses centrally is complex when each location maintains separate or manual records.
Key Issues:

- Time-consuming manual data entry.
- Difficulty in tracking student records across different regions.
- Lack of a unified reporting system for total fees and student counts.

Goal: Create a centralized, digital ERP-style system to streamline operations.

C++ Concepts Applied

struct Student

USER-DEFINED TYPE

01

```
struct Student {
  int    id;
  string name;
  string campus;
  string course;
  double fees;
};
```

Groups 5 fields · Single unit · Pass by value/ref

Bundles all student data into one reusable unit

vector<Student>

DYNAMIC ARRAY · STL

02

```
vector<Student> students;

students.push_back(s);
students.erase(begin + i);
for (auto& s : students)
  printStudent(s);
```

push_back · erase · range-for · size()

Grows/shrinks automatically as students are added or removed

map<string, int>

KEY-VALUE STORE · STL

03

```
map<string, int> count;

for (auto& s : students)
  count[s.campus]++;

// "Mumbai":12, "Pune":8
```

Auto-sorted keys · Campus aggregation

Counts students per campus for the Central Report

fstream (File I/O)

PERSISTENT STORAGE

04

```
// SAVE
ofstream out("students.txt");
out << s.id << "," << s.name;
// LOAD
ifstream in("students.txt");
while (in >> s.id >> comma)
  getline(in, s.name, ',');
```

ofstream save · ifstream load · CSV format

Data survives program exit — written and read each run

Functions

MODULAR DESIGN · SRP

05

```
void addStudent();
void viewByCampus();
void searchByName();
void deleteStudent();
void showReport();
double enterFees();
int getNextId();
```

10+ functions · Single responsibility · Reusable

Each function performs exactly one job in the ERP system

Loops & Control Flow

ITERATION · VALIDATION

06

```
while (true) {   // menu loop
  cin >> fees;
  if (!cin.fail() && fees >= 0)
    return fees;
  cin.clear();
  cin.ignore(max, '\n');
}
```

while(true) · switch/case · cin.clear()

Robust input loops prevent crashes from invalid data

Solution

The Concept: A menu-driven console application developed in C++ that acts as a central hub for all campus data.
Core Features (CRUD):

- **Create:** Add new student profiles with unique IDs.
- **Read:** View all records or filter by specific campus/name.
- **Update/Delete:** Modify or remove records dynamically.

Persistence: Uses File Handling (fstream) to save data to students.txt, ensuring information isn't lost when the program closes.

Working (System Architecture)

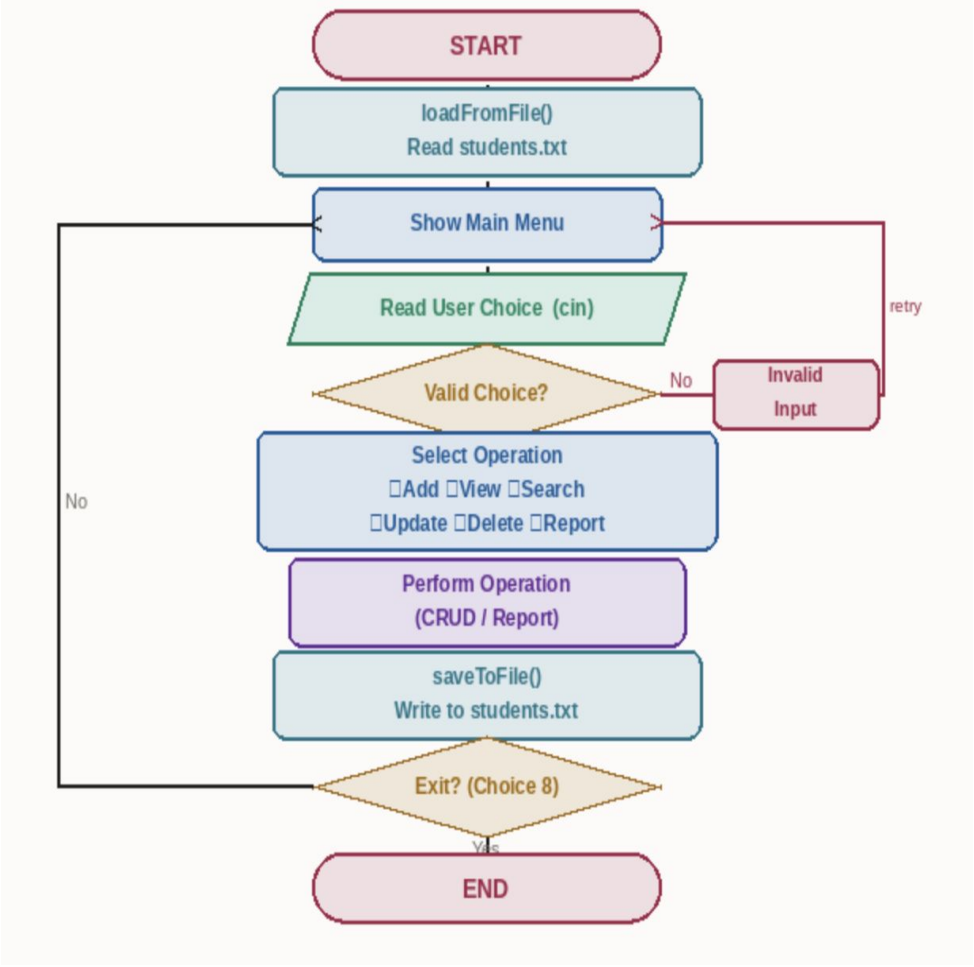
Data Structure: Uses a struct Student to group data (ID, Name, Campus, Course, Fees).
Logic Flow:

1. Storage: A std::vector manages student objects in RAM for fast access.
2. Safety: encode() and decode() functions handle special characters (like commas) to prevent file corruption.
3. Reporting: A std::map<string, int> is used to categorize and count students by their respective campuses.

Working (Process Mind Map)

- Input Validation: The system checks for valid fee amounts and menu choices to prevent crashes.
- Search Algorithm: Uses Linear Search to scan the vector for specific student names using the .find() method.
- File I/O Cycle:
  - Startup: loadFromFile() populates the vector from the disk.
  - Operation: All changes (add/update/delete) trigger saveToFile().
  - Reporting: showReport() calculates totals on demand.

Flowchart:


START
loadFromFile()
Read students.txt
Show Main Menu
Read User Choice (cin)
Valid Choice?
No
Invalid
Input
retry
Select Operation
□Add □View □Search
□Update □Delete □Report
No
Perform Operation
(CRUD / Report)
saveToFile()
Write to students.txt
Exit? (Choice 8)
Yes
END

Conclusion

Project Success: Successfully demonstrated a centralized approach to multi-campus management.

Technical Takeaways:

- Efficiently utilized C++ STL (vectors, maps).
- Implemented robust file handling for permanent storage.

Impact: Reduces manual effort and provides instant access to university-wide data through an ERP-like interface

ITM SKILLS UNIVERSITY

Thank You